Siddhesh Raut (CS23155)

```
import pandas as pd
from sklearn.model_selection import train_test_aplit
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML_Data/Heart_Failure_Prediction_dataset - Heart_Failure_Prediction_dataset.csv"
df=pd.read_csv(path)
df.head(5)
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
df.shape
```

```
(918, 12)
```

```
df.info()
```

Show hidden output

```
df.describe()
```

Show hidden output

```
df.columns
```

```
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
       'HeartDisease'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.isnull().sum()
```

Show hidden output

```
df[df.isnull().any(axis=1)]
```

Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease

```
X = df.iloc[:,3:-1]
y = df['HeartDisease']
```

```
X.head()
```

	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
0	140	289	0	Normal	172	N	0.0	Up
1	160	180	0	Normal	156	N	1.0	Flat
2	130	283	0	ST	98	N	0.0	Up
3	138	214	0	Normal	108	Y	1.5	Flat
4	150	195	0	Normal	122	N	0.0	Up

```
X = pd.get_dummies(X, dtype=int)
```

```
X.shape
```

```
(918, 13)
```

```
X.head(5)
```

	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	Exercis
0	140	289	0	172	0.0	0	1	0	1	
1	160	180	0	156	1.0	0	1	0	1	
2	130	283	0	98	0.0	0	0	1	1	
3	138	214	0	108	1.5	0	1	0	0	
4	150	195	0	122	0.0	0	1	0	1	

```
X.columns
```

```
Index(['RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',
       'RestingECG_LVH', 'RestingECG_Normal', 'RestingECG_ST',
       'ExerciseAngina_N', 'ExerciseAngina_Y', 'ST_Slope_Down',
       'ST_Slope_Flat', 'ST_Slope_Up'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train,y_test = train_test_split(X,y,train_size=0.7,random_state=42)
```

```
X_train.shape,X_test.shape
```

```
((642, 13), (276, 13))
```

```
X_train.head()
```

	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	Exerc
712	100	234	0	156	0.1	0	1	0	1	
477	110	0	1	108	2.0	0	1	0	0	
409	160	0	1	122	0.0	1	0	0	0	
448	160	230	1	105	1.0	0	1	0	0	
838	130	330	1	132	1.8	1	0	0	0	

```
from sklearn.preprocessing import StandardScaler
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

```
array([[-1.77231378,  0.2949892 , -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [-1.23486848, -1.90539005,  1.90228942, ...,  3.93211628,
        -1.02524056, -0.86288193],
```

```
       [ 1.45235803, -1.90539005,  1.90228942, ..., -0.25431598,
         0.97538084, -0.86288193],
       ...,
       [-0.15997788,  0.47365248, -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [ 1.02240179, -1.90539005, -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [ 0.91491273,  1.78071537, -0.52568236, ..., -0.25431598,
         0.97538084, -0.86288193]])
```

```
import tensorflow as tf
```

```
ann = tf.keras.models.Sequential()
```

```
ann.add(tf.keras.layers.Dense(units=9,activation='relu'))
```

```
ann.add(tf.keras.layers.Dense(units=1,activation='sigmoid'))
```

```
ann.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
```

```
ann.fit(X_train,y_train,batch_size=32,epochs=100)
```

Show hidden output

```
y_pred=ann.predict(X_test)
y_pred=(y_pred > 0.5)
y_pred
```

Show hidden output

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
cm=confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
[[ 90  21]
 [ 22 143]]
```

```
print('Accuracy Score')
print(accuracy_score(y_pred,y_test))
```

```
Accuracy Score
0.8442028985507246
```

```
print('Classification Report')
print(classification_report(y_pred,y_test))
```

```
Classification Report
              precision    recall  f1-score   support

       False       0.80      0.81      0.81       111
        True       0.87      0.87      0.87       165

    accuracy                           0.84       276
   macro avg       0.84      0.84      0.84       276
weighted avg       0.84      0.84      0.84       276
```